

AIS Graph Analytics

Architecture, Operations, CI/CD, and Release Documentation

Konstantin Lofichenko

October 2026

Contents

0.1	1. Platform overview	1
0.2	2. Architecture	2
0.3	3. Docker runtime	2
0.4	4. Fresh installation	3
0.5	5. ClickHouse data model	5
0.6	6. dbt	5
0.7	7. Historical AIS processing	6
0.8	8. Daily Airflow orchestration	6
0.9	9. AI enrichment	6
0.10	10. Neo4j and graph analytics	7
0.11	11. Metabase	7
0.12	12. CI	7
0.13	13. Release workflow	8
0.14	14. Published containers	8
0.15	15. Validation and operations	8
0.16	16. Important data-quality semantics	8
0.17	17. Known limitation: midnight visit fragmentation	9
0.18	18. Troubleshooting	9
0.19	19. Safe shutdown and cleanup	9
0.20	20. Repository documentation map	10
0.21	21. Current status and roadmap	10

This document consolidates the current architecture, setup, operational semantics, CI/CD workflow, release process, and known limitations of the AIS Graph Analytics portfolio project.

This document is the operational and architectural companion to the repository root `README.md`. It reflects the current platform after completion of CI/CD and release publishing.

0.1 1. Platform overview

AIS Graph Analytics combines two ingestion modes:

- **Live:** BarentsWatch AIS -> Python producer -> Kafka.
- **Historical:** HAIS GeoParquet -> Airflow -> ClickHouse.

Both feed a canonical ClickHouse AIS history used by dbt analytical models. Neo4j stores graph/current-state structures, GDS produces graph metrics, OpenAI enriches selected daily vessel anomalies, and Metabase provides user-facing analytics.

The design separates responsibilities deliberately:

Concern	System of record / primary engine
Raw event history	ClickHouse
Analytical transformations	dbt on ClickHouse
Current vessel graph state	Neo4j

Concern	System of record / primary engine
Port/community graph analytics	Neo4j GDS + ClickHouse snapshots
Live transport	Kafka
Batch/daily orchestration	Airflow
AI interpretation	External OpenAI enrichment pipeline
Visualization	Metabase
CI/CD	GitHub Actions + GHCR

0.2 2. Architecture

0.2.1 2.1 Live path

BarentsWatch

- > producers/ais
- > Kafka topic ais.positions
 - > ClickHouse Kafka Engine / materialized view -> raw.ais_positions
 - > Kafka Connect Neo4j sink -> current Vessel nodes

The producer uses MMSI as the Kafka key and preserves source AIS fields. ClickHouse keeps event history; the Neo4j sink merges current vessel state by MMSI.

0.2.2 2.2 Historical path

HAIS GeoParquet files

- > Airflow historical ingestion
- > raw.hais_positions
- > dbt staging normalization
- > canonical load macro
- > raw.ais_positions

The HAIS landing layer is intentionally source-faithful. Normalization happens before loading into the canonical AIS table.

0.2.3 2.3 Analytical / graph path

raw.ais_positions

- > dbt vessel models
- > daily features
- > deterministic anomalies
- > AI enrichment history
- > daily enriched vessel view

raw.ais_positions

- > port visit detection
- > Neo4j port graph
- > PageRank + Louvain
- > ClickHouse graph snapshots
- > Metabase

0.3 3. Docker runtime

0.3.1 3.1 Compose profiles

Profile	Main service	Use
none	Kafka, Kafka Connect, ksqlDB, Kafbat UI, ClickHouse, Neo4j	Core runtime
live	AIS producer	Continuous BarentsWatch ingestion

Profile	Main service	Use
batch analytics	Airflow Metabase	Historical and daily analytics BI/dashboarding

0.3.2 3.2 Project-owned images

The release workflow publishes three images to GHCR:

```
ghcr.io/konstantinlofichenko/ais-graph-analytics-airflow
ghcr.io/konstantinlofichenko/ais-graph-analytics-producer
ghcr.io/konstantinlofichenko/ais-graph-analytics-neo4j
```

The Neo4j image is customized because the project adds ClickHouse JDBC support on top of the upstream Neo4j image. Temporary *-ci image names are test artifacts and are not release packages.

0.3.3 3.3 Third-party images

Kafka, ClickHouse, Metabase, ksqldb, Kafbat UI, and other upstream infrastructure are referenced directly from their official registries. They are intentionally not republished as project-owned packages.

0.4 4. Fresh installation

0.4.1 4.1 Repository and environment

```
git clone https://github.com/KonstantinLofichenko/ais-graph-analytics.git
cd ais-graph-analytics
cp .env.example .env
mkdir -p data/hais
```

Configure local secrets in `.env`. At minimum, set ClickHouse and Neo4j credentials. Configure BarentsWatch credentials only when live ingestion is required.

Never commit `.env`, connector passwords, or expanded configuration output that contains secrets.

0.4.2 4.2 Kafka Connect local config

```
cp neo4j/connectors/ais-sink.example.json neo4j/connectors/ais-sink.json
```

Use:

```
bolt://neo4j:7687
```

inside Docker. `localhost` from Kafka Connect refers to the Kafka Connect container itself and is therefore incorrect for Neo4j access.

0.4.3 4.3 Start core services

```
docker compose up -d --wait --wait-timeout 300
docker compose ps
```

0.4.4 4.4 Bootstrap

```
./scripts/bootstrap.sh
```

The bootstrap script:

1. Validates local connector configuration.
2. Creates/verifies Kafka topic `ais.positions`.
3. Applies additive ClickHouse migrations 003 through 010.
4. Applies Neo4j constraints.
5. Creates/updates the Neo4j Kafka sink connector.

Migration `001_replacing_ais_positions.sql` is an existing-installation migration and is intentionally not part of fresh bootstrap.

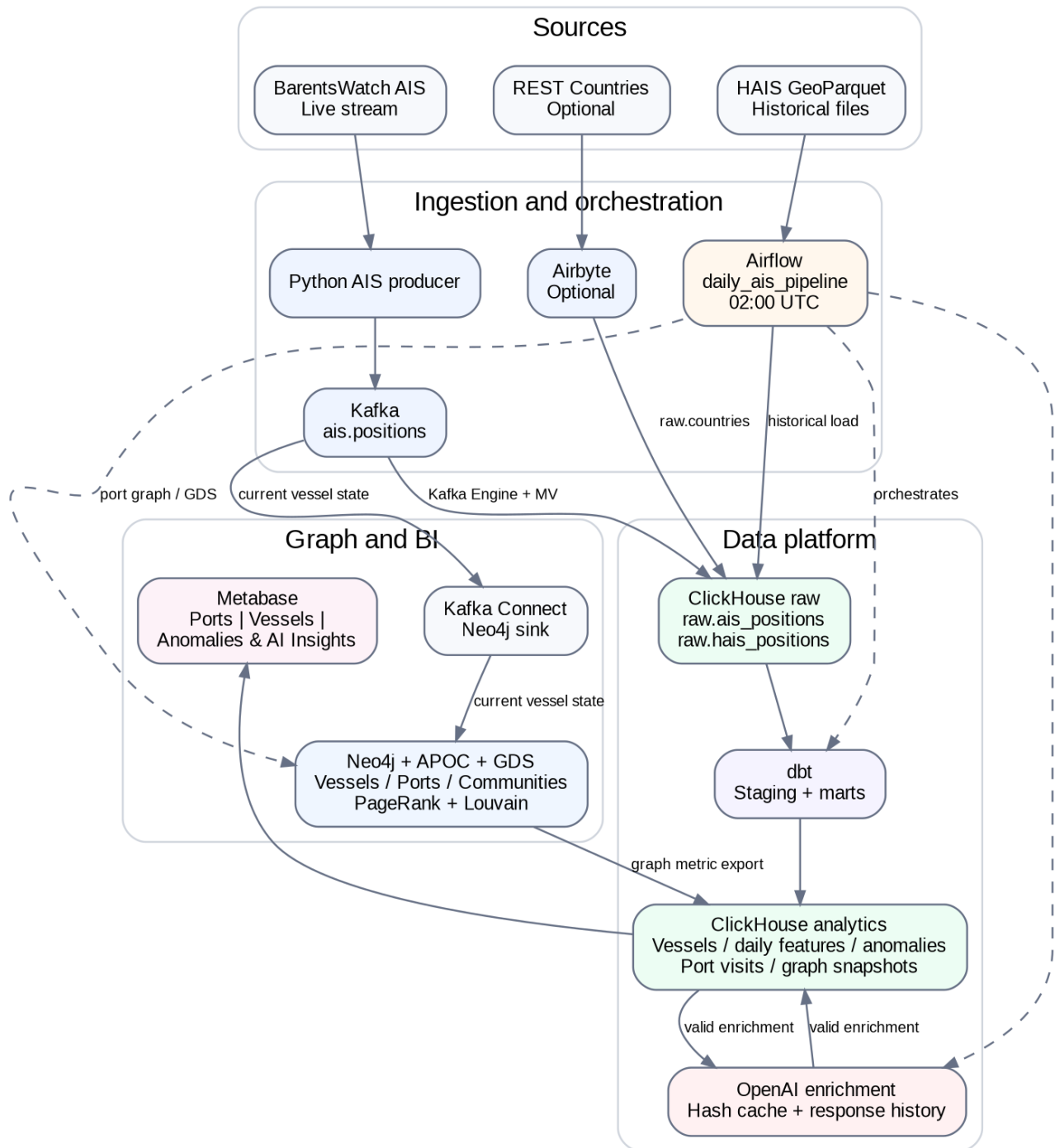


Figure 1: AIS Graph Analytics architecture

0.5 5. ClickHouse data model

0.5.1 5.1 Canonical raw AIS

`raw.ais_positions` is the canonical durable event table. It is optimized for vessel/time access and uses event-time monthly partitions.

Important semantics:

- key analytical access: `(mmsi, msgtime)`;
- version column: `ingested_at`;
- engine: `ReplacingMergeTree(ingested_at)`;
- use `FINAL` for logical validation when replacement convergence matters;
- live and historical sources can overlap, so source coverage should be reviewed before large backfills.

0.5.2 5.2 Historical HAIS

`raw.hais_positions` preserves HAIS source values for auditability. dbt staging normalizes source details before canonical loading.

0.5.3 5.3 Port visits and graph history

`analytics.port_visits` and `analytics.port_graph_metrics` use replacement/tombstone semantics. Stale logical records are marked with newer deleted versions rather than physically removed.

Downstream consumers must use the logical active state (`FINAL` plus the active/deleted filter where required).

0.6 6. dbt

0.6.1 6.1 Environment

```
python3.11 -m venv .venv-dbt
source .venv-dbt/bin/activate
python -m pip install -r dbt/requirements.txt
cp dbt/profiles.yml.example dbt/profiles.yml
export DBT_PROFILES_DIR="$PWD/dbt"
export CLICKHOUSE_USER=default
read -r -s -p 'ClickHouse password: ' DBT_ENV_SECRET_CLICKHOUSE_PASSWORD
printf '\n'
export DBT_ENV_SECRET_CLICKHOUSE_PASSWORD
```

```
dbt deps --project-dir dbt --profiles-dir dbt
dbt parse --project-dir dbt --profiles-dir dbt
dbt debug --project-dir dbt --profiles-dir dbt
```

Current pinned versions are dbt Core 1.12.5 and dbt-clickhouse 1.10.3.

0.6.2 6.2 Important models

Model	Role
<code>vessels</code>	Vessel identity/current analytical state
<code>vessel_daily_features</code>	Daily movement and quality features
<code>vessel_daily_anomalies</code>	Deterministic anomaly selection
<code>vessel_daily_enriched</code>	Daily features + valid current AI enrichment
<code>current_port_visits</code>	Logical current active port visits
<code>port_graph_metrics_enriched</code>	Graph metrics with reference dimensions
<code>port_graph_communities</code>	Community-level reporting table

Static AIS ship-type and navigation-status mappings are dbt seeds, not ad-hoc SQL embedded throughout models.

0.7 7. Historical AIS processing

Historical files are manually placed in `data/hais` with names such as:

`hais_2026-09-01.snappy.parquet`

The historical Airflow ingestion records audit/retry status. After successful source ingestion, dbt staging plus the canonical-load macro populates `raw.ais_positions`.

Date convention:

Operation	Start	End
HAIS file ingestion	Inclusive	Inclusive
Canonical dbt load	Inclusive	Exclusive UTC midnight
Daily analytics	Activity date / UTC window start	Next UTC boundary

Avoid uncontrolled overlapping re-loads: the canonical table can converge duplicate versions, but backfills can still replace attributes such as vessel names/types if the historical source is less complete than live data.

0.8 8. Daily Airflow orchestration

The main DAG is:

`daily_ais_pipeline`

Schedule:

`02:00 UTC daily`

An explicit `dag_run.conf.activity_date` can be used for deterministic reruns/backfills.

The daily orchestration coordinates the analytical chain rather than treating dbt, AI, graph, and dashboard data as independent manual jobs.

Conceptually:

```
canonical AIS ready
-> dbt staging/marts
-> anomaly candidates
-> AI enrichment
-> port visits
-> Neo4j sync / graph analytics
-> graph snapshot export
-> dashboard-ready data
```

Historical backfills should be sequential where graph/current-state updates are involved.

0.9 9. AI enrichment

0.9.1 9.1 Purpose

AI is used to summarize selected anomalous vessel-days, not to replace deterministic feature engineering or anomaly selection.

0.9.2 9.2 Cache and history

`analytics.vessel_ai_enrichment` retains response history including model, prompt version, exact input hash, token counts, response ID, and creation time.

The lookup requires an exact match of the relevant input identity, including date/model/prompt/input hash. This avoids presenting stale enrichment after source features change while preserving historical responses for auditability.

0.9.3 9.3 Bounded calls

`AI_ENRICHMENT_LIMIT` limits **new** API calls after cache hits are removed. CI does not make live OpenAI calls.

0.10 10. Neo4j and graph analytics

0.10.1 10.1 Current state vs history

Neo4j represents the current graph state. ClickHouse stores time-versioned graph snapshots for historical analysis.

0.10.2 10.2 Main relationships

```
(Vessel)-[:VISITED]->(Port)
(Port)-[:CONNECTED_TO]->(Port)
(Port)-[:MEMBER_OF]->(Community)
```

0.10.3 10.3 GDS

The graph workflow includes:

- PageRank for port importance;
- Louvain for community detection;
- deterministic community naming/labels based on representative ports;
- export of graph metrics to ClickHouse.

Community reporting is standardized around human-readable names while retaining the technical community ID for traceability.

0.11 11. Metabase

The main dashboard contains three logical tabs:

1. **Ports**
2. **Vessels**
3. **Anomalies & AI Insights**

The project also includes export/import helpers so Metabase configuration can be treated as code-like project assets rather than only manual UI state.

Dashboard date filters use business `activity_date` semantics instead of exposing technical run IDs where possible.

0.12 12. CI

The CI workflow is intentionally isolated from external services and local developer state.

0.12.1 12.1 Jobs

Job	Validation
Shell and Compose	Shell syntax and all Compose profiles
Pipeline tests	Airflow image + isolated DAG/pipeline tests
Producer	Python producer tests + image build
dbt + ClickHouse	Fresh ClickHouse, init, migrations, CI fixture, seeds, selected model/test graph
Kafka + Neo4j	Kafka topic/produce/consume + custom Neo4j startup/constraints/write/read

0.12.2 12.2 CI source fixture

The dbt integration test creates a deterministic `raw.countries` fixture only for CI because production `raw.countries` is normally owned by optional Airbyte ingestion. The fixture must remain CI-only and should not be added to normal production initialization.

0.12.3 12.3 External calls deliberately excluded

CI does not depend on:

- BarentsWatch;
- OpenAI;
- Airbyte control plane;
- a running local Mac environment;
- production Metabase.

This makes pull-request checks deterministic and reproducible.

0.13 13. Release workflow

A stable semantic tag matching `vMAJOR.MINOR.PATCH` triggers release automation.

Example:

```
git tag -a v1.0.3 -m "Release v1.0.3"
git push origin v1.0.3
```

The workflow:

1. Validates semantic version format.
2. Verifies the release commit belongs to `main`.
3. Runs the reusable CI workflow.
4. Builds the Airflow, producer, and custom Neo4j images.
5. Pushes versioned images to GHCR.
6. Creates a GitHub Release with generated notes.

Stable image tags include:

```
MAJOR.MINOR.PATCH
MAJOR.MINOR
MAJOR
latest
```

Do not move an already-published release tag. Use a new patch/minor/major version instead.

0.14 14. Published containers

```
docker pull ghcr.io/konstantinlofichenko/ais-graph-analytics-airflow:latest
docker pull ghcr.io/konstantinlofichenko/ais-graph-analytics-producer:latest
docker pull ghcr.io/konstantinlofichenko/ais-graph-analytics-neo4j:latest
```

Only project-owned images are published. Third-party images remain referenced from upstream registries.

0.15 15. Validation and operations

Useful project checks:

```
./scripts/status.sh
docker compose ps
docker compose logs --tail 100 airflow
docker compose logs --tail 100 ais-producer
docker compose logs --tail 100 neo4j
```

Kafka topic inspection:

```
docker exec ais-kafka /opt/kafka/bin/kafka-topics.sh \
  --bootstrap-server ais-kafka:29092 --list
```

Logical ClickHouse validation should prefer `FINAL` where `ReplacingMergeTree` replacement semantics matter.

0.16 16. Important data-quality semantics

- Ship type/name coverage depends on the source; source `NULLs` should not be silently presented as fabricated identities.
- AIS navigation status can be noisy; it is supporting evidence rather than a definitive motion label.
- Speed metrics exclude implausible values using the configured validity threshold.

- Stationary percentage is based on observations, not continuous elapsed-time interpolation.
- AI summaries describe the engineered inputs they receive; they do not establish behavior between AIS observations.
- Graph community IDs are technical identifiers; labels/names are user-facing descriptions.

0.17 17. Known limitation: midnight visit fragmentation

Port-visit detection processes bounded UTC windows. If one physical stay crosses midnight, it can appear as separate daily visit fragments because state is not persisted across daily windows.

Impact:

- possible visit overcounting;
- fragmented duration;
- small downstream differences in graph connections/metrics.

Production-grade alternatives are persistent detector state or continuous physical-visit detection followed by daily projection.

0.18 18. Troubleshooting

0.18.1 Container name conflict

```
docker ps -a
```

Remove only obsolete AIS test/manual containers. Do not blindly delete project volumes.

0.18.2 Kafka Connect cannot reach Neo4j

Use `bolt://neo4j:7687`, not `localhost`, inside Docker.

0.18.3 ksqlDB or Kafka Connect healthchecks

The Confluent images used by this project do not rely on `curl` for these healthchecks; the Compose configuration uses TCP checks where appropriate.

0.18.4 Neo4j custom image

The project-owned Neo4j image includes the ClickHouse JDBC JAR and is tested in CI with the plugin environment used by the normal Compose service.

0.18.5 dbt cannot connect

Confirm:

- dbt virtual environment is active;
- `DBT_PROFILES_DIR` points to the project dbt directory;
- `DBT_ENV_SECRET_CLICKHOUSE_PASSWORD` matches the local ClickHouse password;
- host/port are correct for the environment being tested.

0.19 19. Safe shutdown and cleanup

Stop services without deleting named volumes:

```
docker compose --profile live --profile batch --profile analytics down
```

Do **not** add `-v` if data must be retained.

Temporary local CI images can be removed when no longer needed:

```
docker images --format '{{.Repository}}:{{.Tag}}' | grep -- '-ci'
```

Delete only disposable test images, not the project runtime images or upstream dependencies you still use.

0.20 20. Repository documentation map

Path	Purpose
README.md	Project overview and quick start
docs/README.md	Detailed operational/architectural guide
airflow/README.md	Airflow-specific setup and orchestration
dbt/README.md	dbt configuration and transformations
pipelines/port_visits/README.md	Port-visit implementation
pipelines/graph_metrics/README.md	Graph metric export
neo4j/gds/README.md	GDS workflows

0.21 21. Current status and roadmap

Current implementation includes:

- live + historical AIS ingestion;
- canonical ClickHouse history;
- dbt analytical models/tests;
- Airflow scheduled orchestration;
- Neo4j graph analytics;
- deterministic anomaly selection;
- AI enrichment with cache/history;
- Metabase dashboarding;
- five-job CI;
- semantic GitHub Releases;
- GHCR publishing for all project-owned runtime images.

Planned learning extensions such as Flink, MinIO, Iceberg, Trino, and **dbt-trino** are future work and should not be described as part of the current deployed runtime until implemented.